

【第4回】 **DOM**の操作とブラウザでの実行について

1. DOM操作について

1-1. DOMとは

DOMとは「**Document Object Model**」の略で、**HTMLをJavaScriptで操作するための仕組み**のことをいう。
DOMはツリー構造で、以下のように**親要素・子要素・兄弟要素などの関係性**を持つ。

HTML要素

```
<body>
  <div>
    <h1>タイトル</h1>
    <p>本文</p>
  </div>
</body>
```

DOM構造

```
document
├ body
│   └ div
│       ├── h1
│       └ p
```

1. DOM操作について

1-2. DOM操作でできること

DOM操作では、基本的に以下のような操作を行うことができる。

- ・ 要素の取得
- ・ 要素の属性・値の変更・追加・削除
- ・ 要素へのイベントの付与・解除

1-3. DOM操作での要素の取得

DOM操作でHTMLの要素を取得するには、以下の方法がある。

- ・ HTML要素のid属性指定で取得する

`document.getElementById('id名')`

- ・ HTML要素のclass属性指定で取得する

`document.getElementsByClassName('class名')` … 指定のclass名の要素を配列で全て取得する

`document.querySelector('class名')` … 指定のclass名で最初に見つかった要素を取得する

`document.querySelectorAll('class名')` … 指定のclass名の要素を配列で全て取得する

- ・ HTML要素のタグ名指定で取得する

`document.getElementsByTagName('タグ名')` … 指定のタグ名の要素を配列で全て取得する

`document.querySelectorAll('タグ名')` … 指定のタグ名の要素を配列で全て取得する

1. DOM操作について

1-3. DOM操作での要素の取得

例)

```
let txt = document.getElementById('txt') ... id名が[txt]のHTML要素を取得して、変数txtに格納する
let cls = document.getElementsByClassName('cls')
           ... class名が[cls]のHTML要素を全て取得して、変数clsに格納する
let cls = document.querySelector('.cls')
           ... class名が[cls]のHTML要素で最初に見つかった要素を取得して、変数clsに格納する
let cls = document.querySelectorAll('.cls')
           ... class名が[cls]のHTML要素を全て取得して、変数clsに格納する
let p = document.getElementsByTagName('p')
           ... HTML要素内の全てのpタグを取得して、変数pに格納する
let p = document.querySelector('p')
           ... HTML要素内のpタグで最初に見つかった要素を取得して、変数pに格納する
let p = document.querySelectorAll('p')
           ... HTML要素内の全てのpタグを取得して、変数pに格納する
let h1 = document.querySelectorAll('#div h1')
           ... id名[div]内の全てのh1タグを取得して、変数h1に格納する
```

- ※ DOM操作では、**操作したい要素を間違えなく取得することが重要**である
- ※ 取得した要素内のプロパティを指定することで、**取得した要素の各種プロパティ情報を取得する**ことができる
- ※ DOM操作を行うためには、ブラウザが**HTML要素を全て読み込み終わった後に操作する必要がある**

1. DOM操作について

1-4. DOM操作での要素の変更

DOM操作では、指定した要素の属性や要素内のHTML要素・値などを変更することができる。
DOM操作による要素の各種変更は、**WEBページを動的に変更するためには必要不可欠**である。

例)

- テキストを変更する
`document.getElementById('txt').textContent = '変更後テキスト';`
- HTMLを変更する
`document.getElementById('txt').innerHTML = '<strong>太字</strong>';`
- 属性を変更する
`document.getElementById('img').setAttribute('src', 'sample.png');`
- classを変更する
`document.getElementById('txt').classList.add('active');`
`document.getElementById('txt').classList.remove('active');`
- スタイルを変更する
`document.getElementById('txt').style.color = 'red';`
- HTML要素を追加する
`const p = document.createElement('p');`
`p.textContent = 'pタグ追加';`

`document.getElementById('div').appendChild(p);`

1. DOM操作について

【例題1】

Javascript-sample-q1.htmlの要素に対して、DOM操作を使って、以下の操作を行うJavaScriptを実装しなさい。

- ① **HTML**ファイルの読み込み完了時に、現在のタイトルの後ろに「の勉強」の文字を追加して表示する。
- ② **HTML**ファイルの読み込み完了時に、**p**タグの文字を取得して、ダイアログに表示する。
- ③ **HTML**ファイルの読み込み完了時に、**span**タグの文字色を赤色に変更する。
- ④ 入力項目1・入力項目2の背景色をグレー(#F3F3F3)に変更する。
- ⑤ 入力項目2に「入力項目2」というプレースホルダ属性(**placeholder**)を追加する。

2. イベントについて

2-1. イベントとは

イベントとは、ユーザーの操作やブラウザの処理により、**処理を開始するための引き金(トリガー)となる画面の変化のこと**で、「〇〇が△△したら、実行する」という意味合いになる。
一般的にJavaScriptは自動で実行されることは少ないため、イベントごとに処理を実装し、画面の変化に応じて処理を行う。

2-2. イベントの実装方法

イベントを実装する際には、**画面の変化を監視するためのEventListener(イベントリスナー)**を使用して、実装する必要がある。

EventListenerは、JavaScriptに対してではなく、**HTMLの要素に対して実装者がJavaScript内で付与すること**でイベントを検知し、処理を行うことができる。

通常、**デフォルトではHTMLの要素にはEventListenerは付与されていない**ため、必ずJavaScript内で付与しなければ処理を実装することができない。

【EventListenerの付与方法】

[要素].addEventListener('イベント名', [実行する処理] or [呼び出す関数の関数名]);

イベント付与の際の要素の指定は、HTMLのid属性やclass属性を指定することで行う方法が一般的である。

【要素の取得方法】

- ・ id属性で取得 … document.getElementById('id名')
- ・ class属性で取得 … document.getElementsByClassName('class名')
- ・ タグ名で取得 … document.getElementsByTagName('タグ名')

※ class属性やタグ名で取得した際には、HTMLの構造によっては複数の要素が同時に取得される場合がある

2. イベントについて

2-2. イベントの実装方法

※ **headタグでJavaScriptを読み込む場合には、HTMLファイルの読み込み前にJavaScriptが定義されるため、イベントを付与する対象要素が読み込まれておらず、エラーとなる。**
明示的にHTMLファイルの読み込み完了後のイベントを付与し、HTMLファイルの読み込み完了後のイベント内で要素ごとのイベントを個別に定義する必要がある。

例)

[HTMLの要素]

```
<button id='btn'>クリック</button>
```

[JavaScriptでのイベント付与]

```
document.addEventListener('DOMContentLoaded', function() → HTMLの読み込み完了後のイベントを定義
{
    const btn = document.getElementById('btn'); → イベントを付与するHTMLの要素をid指定で取得

    btn.addEventListener('click', function() → 取得したHTML要素へイベントを付与
    {
        alert('クリックされました');
    });
}
```


2. イベントについて

2-3. イベントの種類

イベントには様々な種類があり、使用する**HTML**の要素によっても実装できるイベントが異なる。

イベントの種類	イベントの名前	イベントの概要・発火タイミング
マウスイベント	click	要素のクリックに対応したイベント
	dblclick	要素のダブルクリックに対応したイベント
	mouseover	要素にマウスが乗った際のイベント
	mouseout	要素からマウスが離れた際のイベント
キーイベント	keydown	キーを押した瞬間のイベント
	keyup	キーを離した瞬間のイベント
フォームイベント	change	値が変化した際のイベント
	input	値の入力中のイベント
	submit	データが送信された際のイベント
ページイベント	load	要素の読み込みが完了した際のイベント

2. イベントについて

2-3. イベントの種類

イベントの種類	イベントの名前	イベントの概要・発火タイミング
フォーカスイベント	focus	要素にフォーカスが当たった瞬間のイベント
	blur	要素からフォーカスが外れた瞬間のイベント
	focusin	要素にフォーカスが当たった瞬間のイベント
	focusout	要素からフォーカスが外れた瞬間のイベント

2. イベントについて

2-4. イベントオブジェクトの活用

イベント発生時に実行する**function**の引数として、発生したイベントの内容を取得するイベントオブジェクトを指定することができる。

【イベントオブジェクトの取得方法】

```
[要素].addEventListener('イベント名', function([イベントオブジェクトの名称])  
{  
    ...  
});
```

※ 取得したイベントオブジェクトを使用すると、オブジェクトのプロパティで以下のような情報を取得することができる

- ・ **[イベントオブジェクトの名称].type** … 発生したイベントの種類を取得
- ・ **[イベントオブジェクトの名称].target** … イベントが発生した要素を取得

2-5. イベントリスナーの解除

付与したイベントリスナーを**JavaScript**内で解除することもできる。

ただし、無名関数(イベント発生時に実行する関数を別名で定義せず直接**function**として記述すること)を使用している場合には、イベントの解除を行うことができない。

【**EventListener**の解除方法】

```
[要素].removeEventListener('イベント名', [呼び出す関数の関数名]);
```

2. イベントについて

【例題2】

Javascript-sample-q2.htmlの要素にイベントを実装し、以下のような処理を行えるようにしなさい。

- ① **HTML**ファイルの読込完了時に、「**HTML**の読み込みが完了しました」とダイアログに表示する。
- ② 入力項目**1**からカーソルが離れた際に、「入力項目**1**からフォーカスが離れました」とダイアログに表示する。
- ③ 入力項目**2**に「**a**」を入力した場合にのみ、「**[a]**が入力されました」とダイアログに表示する。
- ④ 入力項目**5**が選択された場合に、「入力項目**5**が選択されました」とダイアログに表示する。
- ⑤ ボタンをクリックされた場合に、「ボタンがクリックされました」とダイアログに表示する。